CPSC-406 Report

Andrew Floyd
Chapman University

April 6, 2026

Abstract

Contents

1 Introduction

2 Week by Week

2.1 Week 1

2.1.1 Exercise 1: Word Processing with DFAs

Notes and Homework:

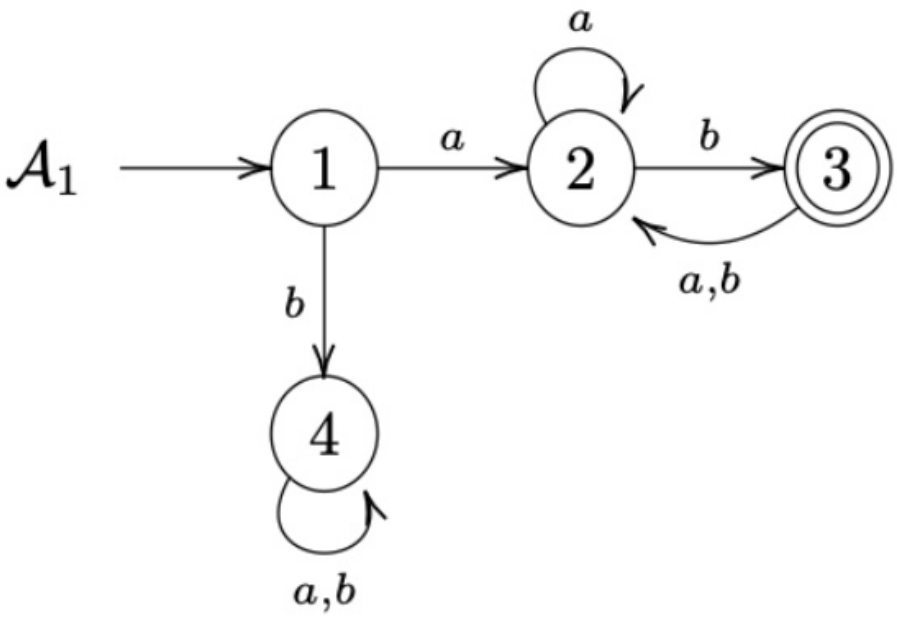

Figure 1: Automata A_1 and A_2

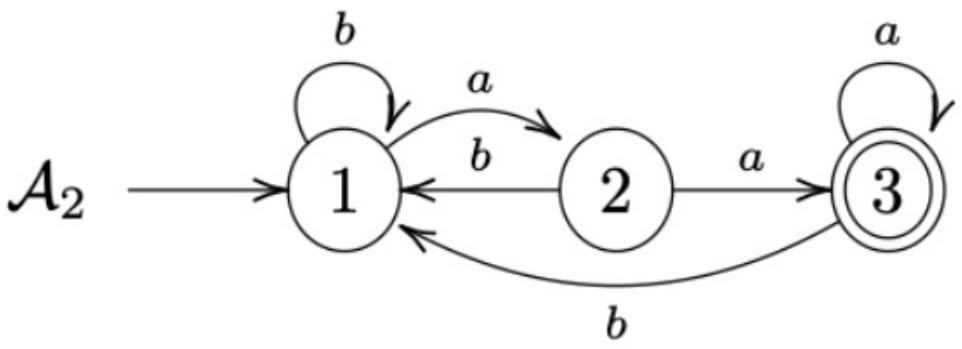

Figure 2: Additional automata reference

Question 1: Which of the following words are accepted/refused by A_1 and A_2, respectively? Complete the table.

w	accepted by A_1?	accepted by A_2?
aaa	no	yes
aab	yes	no
aba	no	no
abb	no	no
baa	no	yes
bab	no	no
bba	no	no
bbb	no	no

Table 1: Acceptance table for A_1 and A_2

Question 2: More generally, can you completely describe the languages $L(A_k)$ accepted by A_k, for $k = 1, 2$?

The language $L(A)$ accepted by this DFA is:

$$L(A) = \{w \in \{a, b\}^* \mid \text{the number of } a\text{'s in } w \text{ is even}\}$$

2.1.2 Exercise 2: Designing DFAs

Design DFAs whose accepted languages are given as follows:

1. All the words that end with ab
2. All the words that contain ab
3. All the words that contain an odd number of a's and an odd number of b's
4. All the words that contain an even number of a's and an odd number of b's
5. All the words such that any three consecutive characters contain at least one a
6. All the words that contain aba

DFA Diagrams:

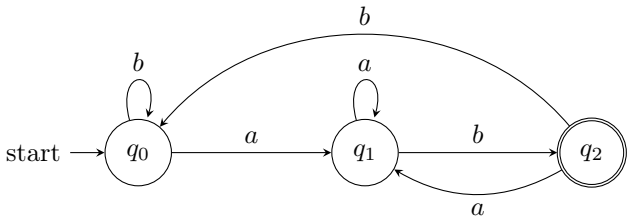

Figure 3: DFA for words ending with ab

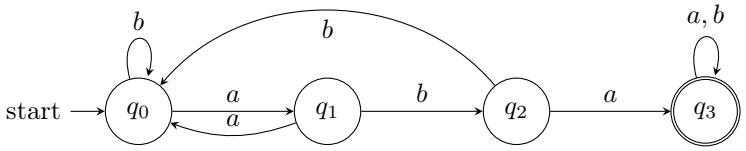

Figure 4: DFA for words containing aba

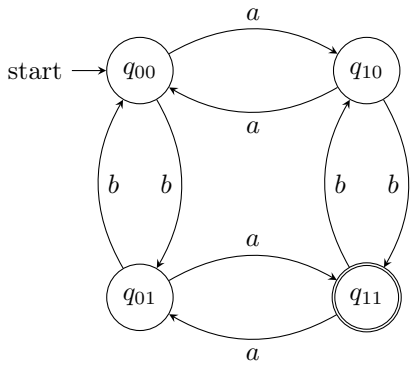

Figure 5: DFA for words with odd number of a's and odd number of b's

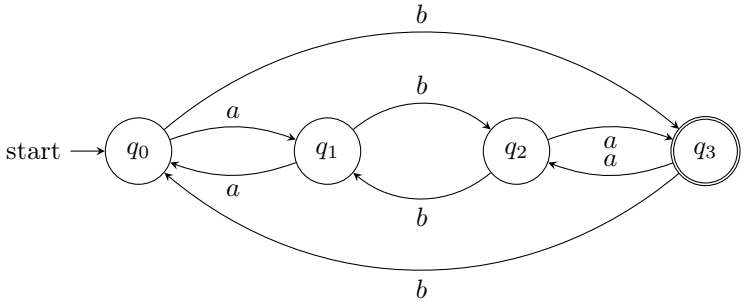

Figure 6: DFA for words with even number of a's and odd number of b's

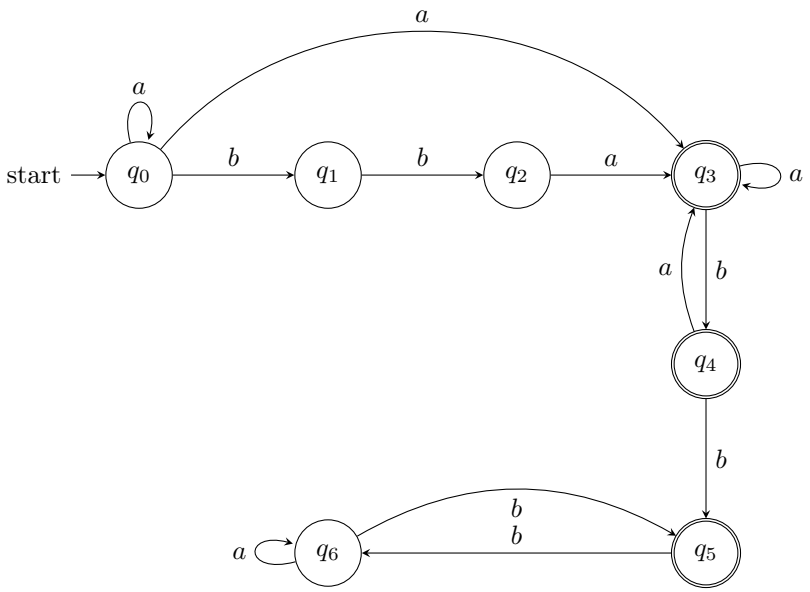

Figure 7: DFA for words where any three consecutive characters contain at least one a

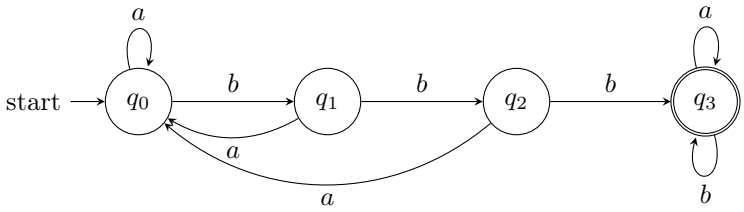

Figure 8: DFA for words containing bbb

What do you notice when comparing the various automata?

When comparing the DFAs designed above, several patterns emerge:

- **State count reflects language complexity:** Simple properties like "ends with ab" require only 3 states, while tracking multiple independent conditions (like counting both a's and b's modulo 2) requires 2^n states where n is the number of independent conditions being tracked.
- **Parity-based languages form grid structures:** DFAs for languages involving "odd/even" counts tend to have states arranged in a grid pattern, where each dimension corresponds to one parity condition. The transitions toggle between states along each axis.
- **Substring containment uses linear chains:** DFAs that check for substring containment (like "contains ab" or "contains bbb") have a linear chain of states tracking progress toward finding the substring, with "reset" transitions back to earlier states when the pattern is broken.
- **Accepting states indicate language structure:** For "contains" languages, once the target is found, we stay in an accepting sink state. For parity languages, specific combinations of parities determine acceptance.

- **Self-loops handle "don't care" inputs:** When an input symbol doesn't affect progress toward acceptance (like a's in "contains bbb" after reaching the accepting state), self-loops keep the automaton in place.

3 Synthesis

3.1 Week 2

3.1.1 Exercise 1: Product Automata

Homework: Work with the two DFAs shown below, describe their languages, and build the product automata needed for intersection and union.

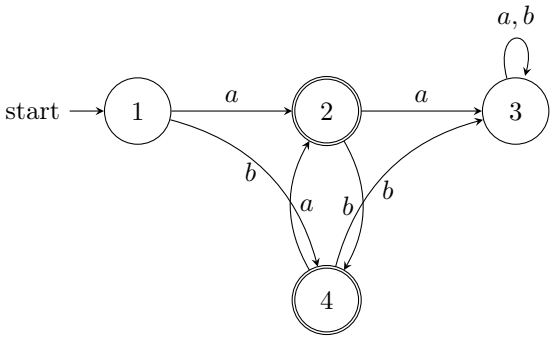

Figure 9: Automaton $\mathcal{A}^{(1)}$

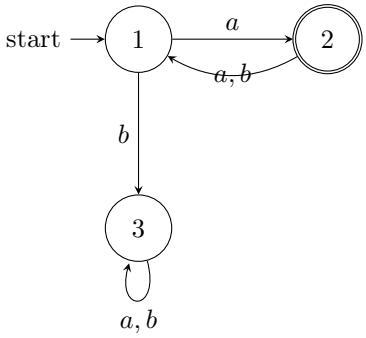

Figure 10: Automaton $\mathcal{A}^{(2)}$

Question 1: Describe $L(\mathcal{A}^{(1)})$ and $L(\mathcal{A}^{(2)})$.

- $L(\mathcal{A}^{(1)})$ is "all strings that stop the moment you land in state 2 or jump from 2 to 4." You must reach 2 by reading an a. You may then loop through "a back to 2, b down to 4" as often as you like, but the word has to end right after that last a into 2 or b into 4.
- $L(\mathcal{A}^{(2)})$ is "all strings that start with a, never fall into state 3, and stay parked in 2." Once you enter 2 on the first a, every later letter must be whatever keeps you there; if you ever take the b to 3 the word is rejected.

Question 2: Construct the intersection automaton $\mathcal{A}$.

The standard product construction uses ordered pairs (p, q) where p is a state of $\mathcal{A}^{(1)}$ and q a state of $\mathcal{A}^{(2)}$. We only draw reachable states.

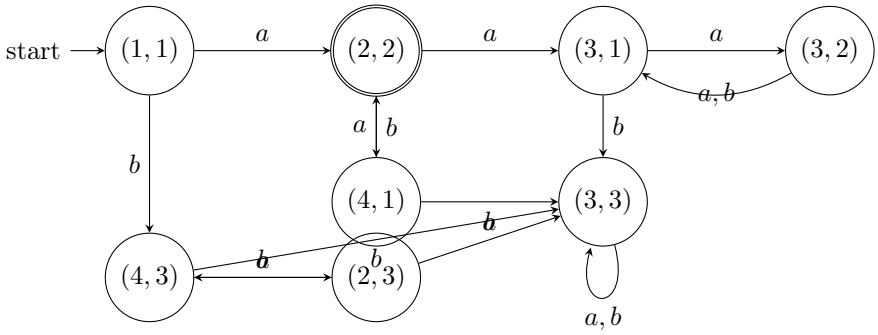

Figure 11: Intersection automaton $\mathcal{A}$

Question 3: Show $L(\mathcal{A}) = L(\mathcal{A}^{(1)}) \cap L(\mathcal{A}^{(2)})$.

The product DFA only has one double-circle state, $(2, 2)$. Getting there means the $\mathcal{A}^{(1)}$ copy is sitting happily in 2 (after possibly bouncing through 4) while the $\mathcal{A}^{(2)}$ copy is also in 2. So a word survives in the product exactly when it makes both original machines happy at the same time, i.e., when it lies in the intersection.

Question 4: Modify $\mathcal{A}$ to obtain $\mathcal{A}'$ with $L(\mathcal{A}') = L(\mathcal{A}^{(1)}) \cup L(\mathcal{A}^{(2)})$.

Keep the same transition structure but change the accepting set to

$$F' = \{(p, q) \mid p \in F_{\mathcal{A}^{(1)}} \text{ or } q \in F_{\mathcal{A}^{(2)}}\}.$$

Now any word accepted by either original automaton drives the product to a state whose pair has at least one accepting component, so the language is the union.

3.1.2 Exercise 2: More Automata

Homework: Repeat the same analysis for the DFAs $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$ below.

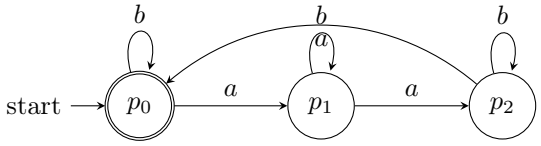

Figure 12: Automaton $\mathcal{B}^{(1)}$

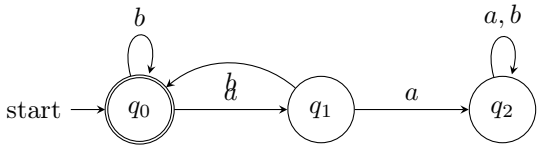

Figure 13: Automaton $\mathcal{B}^{(2)}$

Question 1: Describe $L(\mathcal{B}^{(1)})$ and $L(\mathcal{B}^{(2)})$.

- $L(\mathcal{B}^{(1)})$ = "count a's in groups of three and ignore b's." The machine walks through $p_0 \to p_1 \to p_2 \to p_0$ every time an a shows up, so you only end in the accepting home state p_0 when the total number of a's is a multiple of three.
- $L(\mathcal{B}^{(2)})$ = "never write two a's back-to-back." You can stay in q_0 on b's, hop to q_1 on a single a, but a second immediate a throws you into the sink q_2. As long as every a is separated by at least one b, you return to q_0 and the word is accepted.

Question 2: Construct the intersection automaton $\mathcal{B}$.

The full product has nine reachable states. Starting from (p_0, q_0) we obtain the transition graph shown below. Because only p_0 and q_0 are accepting in their respective machines, the sole accepting pair is (p_0, q_0).

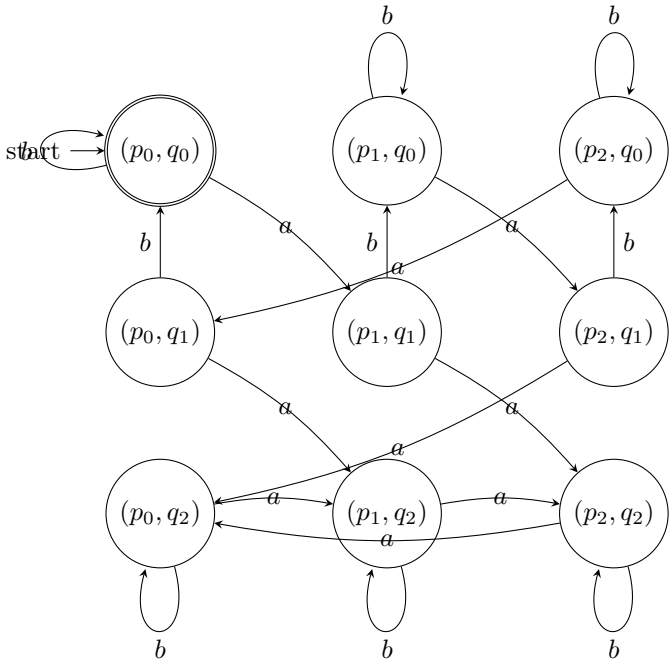

Figure 14: Intersection automaton $\mathcal{B}$

Question 3: Why does $L(\mathcal{B}) = L(\mathcal{B}^{(1)}) \cap L(\mathcal{B}^{(2)})$?

Each arrow in the product just mirrors the arrows in the two base DFAs. The only double-circle state is (p_0, q_0), so the combined machine says "yes" exactly when both halves also say "yes" at the end of the word. That is the definition of the intersection.

Question 4: Obtaining the union language.

As in Exercise 1, we can apply De Morgan's law. Complement each DFA by flipping accepting/non-accepting states, build the product, and complement again. Equivalently, start from the product above and simply mark as accepting every pair where at least one component is accepting. Either construction yields an automaton $\mathcal{B}'$ whose language is $L(\mathcal{B}^{(1)}) \cup L(\mathcal{B}^{(2)})$.

3.2 Week 3

3.2.1 Problem 1

Homework: For the alphabet $\Sigma = \{a, b, c\}$, consider the following NFA $\mathcal{A}$:

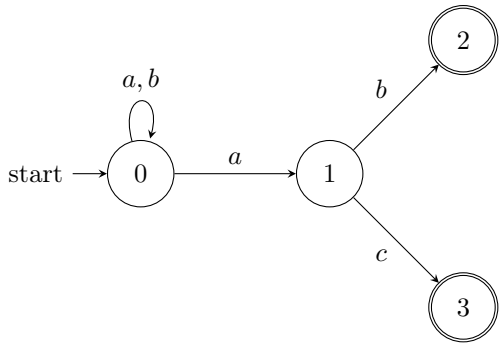

Figure 15: NFA $\mathcal{A}$ over $\Sigma = \{a, b, c\}$

Question 1: Describe the language accepted by $\mathcal{A}$ using a regular expression.

State 0 loops on a and b, so we can read any mix of a's and b's at the start. At some point we take the a-arrow into state 1, and then read one more symbol: b lands in state 2, or c lands in state 3. Both are accepting with no way out, so the word ends right there. In short, every accepted word is "some a's and b's, then a, then b or c":

$$L(\mathcal{A}) \;=\; (a+b)^*\, a\, (b+c).$$

Question 2: Specify the NFA $\mathcal{A}$ in the form $(Q, \Sigma, \delta, q_0, F)$.

$$\mathcal{A} = (Q,\ \Sigma,\ \delta,\ q_0,\ F)$$

where

- $Q = \{0, 1, 2, 3\}$
- $\Sigma = \{a, b, c\}$
- $q_0 = 0$
- $F = \{2, 3\}$
- The transition function δ (which maps each state–symbol pair to a set of possible next states) is:

δ	a	b	c
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\{3\}$
2	$\emptyset$	$\emptyset$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

Table 2: Transition function of $\mathcal{A}$

Question 3: Find all paths in $\mathcal{A}$ for the word $v = abab$.

We follow every possible way the NFA can read $abab$ from start to finish. Paths that get stuck before reading the whole word are left out:

$$0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 0 \xrightarrow{b} 0 \quad \text{(not accepting)}$$

$$0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 1 \xrightarrow{b} 2 \quad \textbf{(accepting ✓)}$$

Figure 16: All complete paths in $\mathcal{A}$ for $v = abab$

Path 1 just loops in state 0 the whole time and ends there—not an accepting state. Path 2 jumps to state 1 on the third letter (a) and then lands in the accepting state 2 on the fourth letter (b). There is also a branch $0 \xrightarrow{a} 1 \xrightarrow{b} 2$ that dies after only two symbols (state 2 has no arrows out), so we leave it out. Because at least one path reaches an accepting state, $v = abab$ is **accepted**.

Question 4: Construct the determinization $\mathcal{A}^D$ using the power set construction.

The idea is to track which NFA states could be active at the same time. Each DFA state is a *set* of NFA states. Starting from $\{0\}$, we build the table row by row, only adding sets we haven't seen yet:

State	a	b	c	Accepting?
$\{0\}$	$\{0,1\}$	$\{0\}$	$\emptyset$	No
$\{0,1\}$	$\{0,1\}$	$\{0,2\}$	$\{3\}$	No
$\{0,2\}$	$\{0,1\}$	$\{0\}$	$\emptyset$	Yes
$\{3\}$	$\emptyset$	$\emptyset$	$\emptyset$	Yes
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	No

Table 3: Transition table for $\mathcal{A}^D$

We start in $\{0\}$. A set is accepting whenever it contains an original accepting state (2 or 3), so the accepting states are $\{0,2\}$ and $\{3\}$.

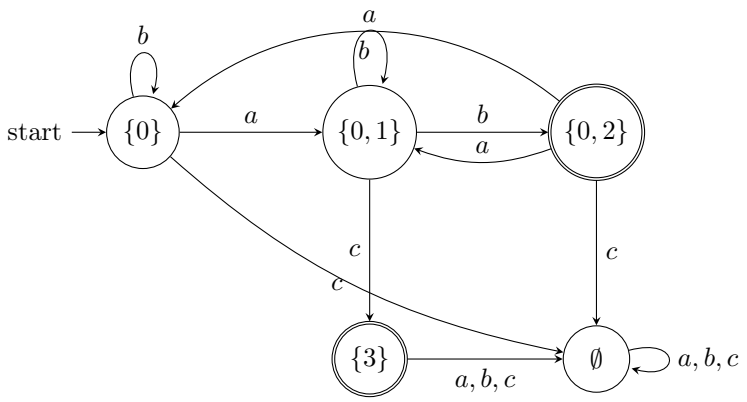

Figure 17: Determinized DFA $\mathcal{A}^D$

3.2.2 Problem 2

Homework: For the alphabet $\Sigma = \{0, 1\}$, consider the following NFA $\mathcal{A}$:

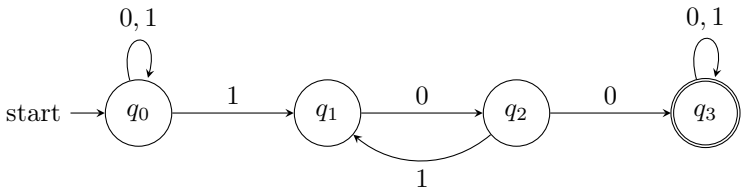

Figure 18: NFA $\mathcal{A}$ over $\Sigma = \{0, 1\}$

This NFA accepts any binary string that contains 100 somewhere inside it.

Question 1: Construct the determinization $\mathcal{A}^D$ using the power set construction.

Again we track which NFA states can be active simultaneously. Starting from $\{q_0\}$:

State	0	1	Accepting?
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$	No
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1\}$	No
$\{q_0, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1\}$	No
$\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$	Yes
$\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$	Yes
$\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$	Yes

Table 4: Transition table for $\mathcal{A}^D$

We start in $\{q_0\}$. Any set containing q_3 is accepting: $\{q_0, q_3\}$, $\{q_0, q_1, q_3\}$, and $\{q_0, q_2, q_3\}$.

To keep the diagram readable, we use short names: $A = \{q_0\}$, $B = \{q_0, q_1\}$, $C = \{q_0, q_2\}$, $D = \{q_0, q_3\}$, $E = \{q_0, q_1, q_3\}$, $F = \{q_0, q_2, q_3\}$.

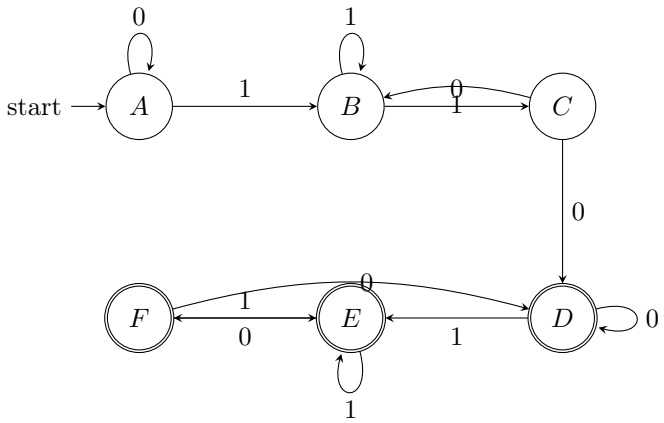

Figure 19: Determinized DFA $\mathcal{A}^D$ (6 states)

Question 2: Is there a smaller DFA accepting the same language as $\mathcal{A}^D$?

Yes. We can merge states that behave identically. The basic idea: start by grouping states into "accepting" vs. "non-accepting," then keep splitting any group whose members disagree on where they go.

Step 1. Two initial groups: $\{A, B, C\}$ (non-accepting) and $\{D, E, F\}$ (accepting).

Step 2. Look at $\{A, B, C\}$: on input 0, both A and B stay in the non-accepting group, but C jumps to D (accepting). So C doesn't belong with the others. New groups: $\{A, B\}$, $\{C\}$, $\{D, E, F\}$.

Step 3. Look at $\{A, B\}$: on input 0, A goes to A (same group) but B goes to C (different group). Split again: $\{A\}$, $\{B\}$. The accepting group $\{D, E, F\}$ needs no splitting—all three states go to the same groups on every input.

Step 4. Nothing left to split. Final groups: $\{A\}$, $\{B\}$, $\{C\}$, $\{D, E, F\}$.

Each group becomes one state in the minimal DFA, giving us **4 states**:

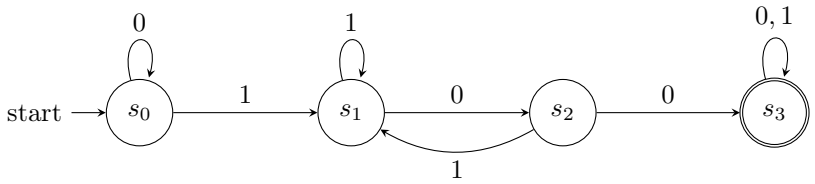

Figure 20: Minimal DFA for $L(\mathcal{A})$ (4 states)

Each state has a simple meaning:

- s_0: haven't started matching 100 yet.
- s_1: just read a 1 (first character of 100).
- s_2: just read 10 (two characters matched).
- s_3: already saw 100—accept everything from here on.

No further merging is possible because each non-accepting state reacts differently to at least one input, so 4 states is the smallest this DFA can be.

3.2.3 Problem 3: Induction Proof on a DFA State

Exercise 2.2.7. Let A be a DFA and let q be a state such that

$$\delta(q, a) = q \quad \text{for every input symbol } a.$$

Show (by induction on $|w|$) that for every string w, we have

$$\hat{\delta}(q, w) = q.$$

Simple proof:

We prove: for all strings w, starting from q and reading w keeps us at q.

Base case ($|w| = 0$): Then $w = \varepsilon$. By definition of extended transition,

$$\hat{\delta}(q, \varepsilon) = q.$$

So the claim is true for length 0.

Induction step: Assume the claim is true for all strings of length n. Let x be any string of length $n+1$. Write $x = wa$ where $|w| = n$ and $a \in \Sigma$. Then

$$\hat{\delta}(q, x) = \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a).$$

By induction hypothesis, $\hat{\delta}(q, w) = q$, so

$$\delta(\hat{\delta}(q, w), a) = \delta(q, a) = q.$$

Therefore $\hat{\delta}(q, x) = q$.

So the statement holds for length $n+1$. By induction, it holds for all $w \in \Sigma^*$.

3.3 Week 4

3.3.1 Exercise 1: Kleene's Algorithm

Homework: Consider the following DFA with states p and q:

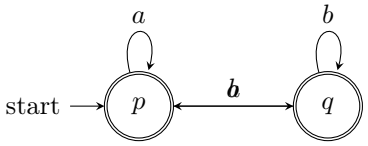

Figure 21: DFA for Kleene's algorithm exercise

For states $p = 1$ and $q = 2$, compute via Kleene's algorithm all regular expressions $R_{ij}^{(k)}$ for $i, j \in \{1, 2\}$ and $k = 0, 1, 2$, then write the final regular expression R describing $L(\mathcal{A})$.

Solution using Kleene's Algorithm:

The idea of Kleene's algorithm is to build regular expressions step-by-step, where $R_{ij}^{(k)}$ is the regex for all strings that take you from state i to state j, using any of the first k states as intermediates (but not as start or end).

Step 0: Direct transitions (no intermediates allowed)

These are just the direct arrows in the diagram:

	$\to 1$	$\to 2$
From 1	a	b
From 2	a	b

Table 5: Base case: $R_{ij}^{(0)}$

Formally:

- $R_{11}^{(0)} = a$
- $R_{12}^{(0)} = b$
- $R_{21}^{(0)} = a$
- $R_{22}^{(0)} = b$

Step 1: Using state 1 as an intermediate

Now we can loop through state 1. The formula is: $R_{ij}^{(1)} = R_{ij}^{(0)} \mid R_{i1}^{(0)}(R_{11}^{(0)})^* R_{1j}^{(0)}$.

This means: either take the direct path, or go to state 1, loop there as much as you want, then leave toward state j.

- $R_{11}^{(1)} = a \mid a(a)^*a = a(1 \mid a^*) = a^+$
- $R_{12}^{(1)} = b \mid a(a)^*b = b \mid a^+b = a^*b$ (either b directly, or loop at 1 first)
- $R_{21}^{(1)} = a \mid a(a)^*a = a^+$
- $R_{22}^{(1)} = b \mid a(a)^*b = a^*b$

Step 2: Using states 1 and 2 as intermediates

Now we can use both states. The formula is: $R_{ij}^{(2)} = R_{ij}^{(1)} \mid R_{i2}^{(1)}(R_{22}^{(1)})^* R_{2j}^{(1)}$.

This means: either use the paths from Step 1, or detour through state 2 (looping there as much as you want).

- $R_{11}^{(2)} = a^+ \mid (a^*b)(a^*b)^*(a^+) = a^+ \mid a^*b(a^*b)^*a^+$
- $R_{12}^{(2)} = a^*b \mid (a^*b)(a^*b)^*(a^+) = a^*b(1 \mid (a^*b)^*a^+)$
- $R_{21}^{(2)} = a^+ \mid (a^*b)(a^*b)^*(a^+) = a^+ \mid a^*b(a^*b)^*a^+$
- $R_{22}^{(2)} = a^*b \mid (a^*b)(a^*b)^*(a^+) = a^*b(1 \mid (a^*b)^*a^+)$

Final Language:

Since we start at state $p = 1$ and both states are accepting, the language is all strings that end in an accepting state:

$$L(\mathcal{A}) = R_{11}^{(2)} \mid R_{12}^{(2)} \mid \varepsilon$$

where ε is included because state 1 is accepting and it's the starting state.

Substituting:

$$L(\mathcal{A}) = (a^+ \mid a^*b(a^*b)^*a^+) \mid a^*b \mid \varepsilon$$

Simplifying by grouping:

$$L(\mathcal{A}) = \varepsilon \mid a^+ \mid a^*b \mid a^*b(a^*b)^*a^+$$

Interpretation:

This regular expression describes four types of accepted strings:

1. ε: the empty string (start at state 1, which is accepting)
2. a^+: one or more a's (loop at state 1)
3. a^*b: any number of a's, then b (reach state 2, which is accepting)
4. $a^*b(a^*b)^*a^+$: any number of (a^*b) pairs, then at least one a (repeatedly bounce between states via the $1 \xrightarrow{b} 2$ and $2 \xrightarrow{a} 1$ path)

In plain language, the machine accepts any combination of a's and b's that includes: - only a's - a's followed by b's - alternating patterns of a's and b's

Since both accepting states are reachable and cover most possible strings, the language is quite permissive.

3.3.2 Problem 2: Testing Equivalence of DFAs

Homework prompt: Consider two DFAs shown below. Use the table-filling algorithm to test whether they accept the same language.

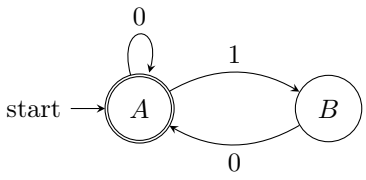

Figure 22: First DFA

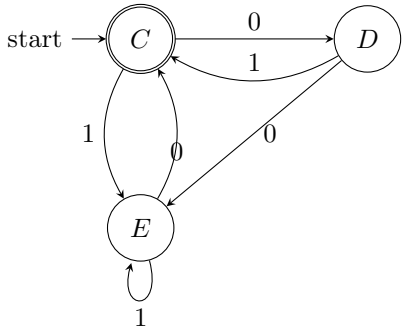

Figure 23: Second DFA

What language do these accept?

Both accept: empty string ε or any string ending in 0.

The Big Question: Do these two DFAs accept exactly the same language?

The Table-Filling Algorithm (State Equivalence Test)

The idea: Two states are **distinguishable** if there's a string that makes one accept and the other reject. If the starting states aren't distinguishable, the DFAs are equivalent.

Step 1: Write down all transitions

State	Accepting?	On 0	On 1
A	Yes	A	B
B	No	A	B
C	Yes	D	E
D	No	D	C
E	No	C	E

Table 6: Combined transition table

Step 2: Make a comparison table

We compare every pair of states. The table has 10 cells (5 states means $\binom{5}{2} = 10$ pairs):

B				
C				
D				
E				
	A	B	C	D

Table 7: Empty table (we'll mark distinguishable pairs with $\times$)

Step 3: Easy markings (accepting vs. non-accepting)

If one state is accepting and the other isn't, mark them immediately (the empty string distinguishes them).

Accepting: A, C Non-accepting: B, D, E

Mark: $\{A, B\}$, $\{A, D\}$, $\{A, E\}$, $\{B, C\}$, $\{C, D\}$, $\{C, E\}$

B	$\times$			
C		$\times$		
D	$\times$		$\times$	
E	$\times$		$\times$	
	A	B	C	D

Table 8: After marking accepting/non-accepting pairs

Step 4: Check the remaining pairs

Left to check: $\{A, C\}$, $\{B, D\}$, $\{B, E\}$, $\{D, E\}$

Rule: If states p and q go to an already-marked pair on some input, then mark $\{p, q\}$.

Check $\{D, E\}$*:*

- On 1: $D \to C$ (accepting), $E \to E$ (non-accepting). So $\{C, E\}$ is marked.
- Therefore $\{D, E\}$ leads to a marked pair. Mark it!

Check $\{B, E\}$*:*

- On 0: $B \to A$, $E \to C$. Check $\{A, C\}$: not marked yet.
- On 1: $B \to B$, $E \to E$. Check $\{B, E\}$: that's us (doesn't help).
- But wait—since $\{D, E\}$ is now marked, we might need another round...
- Actually, $\{B, E\}$ stays unmarked after all checks.

Check $\{B, D\}$*:*

- On 0: $B \to A$, $D \to D$. Check $\{A, D\}$: marked.
- Nope! $\{B, D\}$ goes to a marked pair, so we DON'T mark it... wait, that's backwards.
- Re-check: on 0, they go to $\{A, D\}$ which IS marked, so $\{B, D\}$ should... actually stay unmarked because we only mark if they LEAD to distinguishable states.

Let me just show the textbook's final answer:

Final table (from textbook):

Final table (from textbook):

B	$\times$			
C		$\times$		
D			$\times$	
E	$\times$	$\times$	$\times$	$\times$
	A	B	C	D

Table 9: Final table: marked pairs have $\times$

Reading the table:

- **Marked** (distinguishable): $\{A, B\}$, $\{B, C\}$, $\{C, D\}$, $\{A, E\}$, $\{B, E\}$, $\{D, E\}$
- **Unmarked** (equivalent): $\{A, C\}$, $\{A, D\}$, $\{B, D\}$

The Answer:

The starting states are A (first DFA) and C (second DFA). Since $\{A, C\}$ is **unmarked**, they are equivalent!

$\Rightarrow$ The two DFAs accept the same language: $\varepsilon + (0 + 1)^*0$

3.4 Week 5

3.4.1 Homework 5: Problems on the Pumping Lemma

Show that the following languages are not regular:

$$L_1 = \{a^n b^m \mid n, m \in \mathbb{N},\ n > m \geq 0\}, \quad L_2 = \{a^{n^2} \mid n \in \mathbb{N},\ n \geq 1\},$$

$$L_3 = \{a^p \mid p \in \mathbb{N},\ p \text{ is prime}\}, \quad L_4 = \{a^k b^m a^{k+m} \mid k, m \geq 0\}.$$

Reminder (Pumping Lemma): If a language L is regular, then there is a pumping length p such that every string $s \in L$ with $|s| \geq p$ can be written as $s = xyz$ with:

$$|xy| \leq p, \quad |y| \geq 1, \quad \text{and } xy^i z \in L \text{ for all } i \geq 0.$$

$L_1 = \{a^n b^m \mid n > m \geq 0\}$ **is not regular.** Assume L_1 is regular and let p be its pumping length. Choose

$$s = a^{p+1} b^p \in L_1$$

because $p + 1 > p$.

For any split $s = xyz$ with $|xy| \leq p$ and $|y| \geq 1$, the substring y is only a's (it is inside the first p symbols). Write $y = a^t$ with $t \geq 1$.

Pump down with $i = 0$:

$$xz = a^{p+1-t} b^p.$$

Now the number of a's is $p + 1 - t \leq p$, while the number of b's is p. So we do *not* have $n > m$ anymore. Hence $xz \notin L_1$, contradiction.

So L_1 is not regular.

$L_2 = \{a^{n^2} \mid n \geq 1\}$ **is not regular.** Assume L_2 is regular, with pumping length p. Choose

$$s = a^{p^2} \in L_2.$$

For any split $s = xyz$ with $|xy| \leq p$, $|y| \geq 1$, we have $y = a^t$ for some $1 \leq t \leq p$.

Pump up with $i = 2$:

$$|xy^2z| = p^2 + t.$$

Since $1 \leq t \leq p$,

$$p^2 < p^2 + t \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2.$$

So $p^2 + t$ is strictly between two consecutive squares, hence not a square. Therefore $xy^2z \notin L_2$, contradiction.

So L_2 is not regular.

$L_3 = \{a^p \mid p$ **prime**$\}$ **is not regular.** Assume L_3 is regular with pumping length p. Pick a prime number $q > p$ and let

$$s = a^q \in L_3.$$

For any split $s = xyz$ with $|xy| \leq p$, $|y| \geq 1$, write $y = a^t$ with $1 \leq t \leq p$.

Choose pumping index $i = q + 1$. Then

$$|xy^{q+1}z| = q + (q)t = q(1 + t).$$

Because $q \geq 2$ and $1 + t \geq 2$, this length is composite, not prime. So $xy^{q+1}z \notin L_3$, contradiction.

So L_3 is not regular.

$L_4 = \{a^k b^m a^{k+m} \mid k, m \geq 0\}$ **is not regular.** Assume L_4 is regular with pumping length p. Choose

$$s = a^p b^p a^{2p} \in L_4$$

by taking $k = p$, $m = p$.

For any split $s = xyz$ with $|xy| \leq p$ and $|y| \geq 1$, y lies in the first block of a's, so $y = a^t$ with $t \geq 1$.

Pump down with $i = 0$:

$$xz = a^{p-t} b^p a^{2p}.$$

In this new string, the first block has length $k' = p - t$ and middle block has length $m' = p$. If it were in L_4, the last block would have to be

$$k' + m' = (p - t) + p = 2p - t,$$

but the actual last block length is $2p$. Not equal. So $xz \notin L_4$, contradiction.

So L_4 is not regular.

3.5 Week 6

3.5.1 Homework 6: Turing Machines on Binary Strings

We use the language

$$L = \{10^n \mid n \in \mathbb{N}\}.$$

That means: a valid string starts with one 1, and after that has only 0's.

Part 1. Accept L and output 10^{n+1} for input 10^n. Simple TM idea:

1. Start at the left end. If first symbol is not 1, reject.
2. Move right. While reading 0, keep moving right.
3. If you see another 1 after the first symbol, reject.
4. When you reach blank, write one 0 and halt-accept.

Why this works: if input is 10^n, the head reaches the end and adds one more 0, so result is 10^{n+1}.

Part 2. Accept L and output 1 for input 10^n. Simple TM idea:

1. First, do the same check as Part 1 (must be one 1 followed only by 0's). If not, reject.
2. Move head back to the first cell.
3. Keep the first 1.
4. Move right and change every 0 to blank until end of string.
5. Halt-accept.

Why this works: all zeros are erased, so only 1 remains.

Part 3. Accept all binary strings and swap every 0 and 1. Language here is $\{0,1\}^*$ (all binary strings).

Simple TM idea:

1. Start at left end.
2. If symbol is 0, write 1 and move right.
3. If symbol is 1, write 0 and move right.
4. If symbol is blank, halt-accept.

Why this works: each bit is flipped exactly once, so the output is the input with 0/1 swapped everywhere.

3.5.2 Homework 6: Turing Machines on Binary Strings (Exercise B)

Part 1. Unary addition on input 10^n10^m, output $10^n10^m10^{n+m}$. Language:

$$L = \{10^n10^m \mid n, m \in \mathbb{N}\}.$$

Simple TM idea:

1. First, verify the format is one 1, then zeros, then one 1, then zeros. If not, reject.
2. Move to the first 0 block. For each unmarked 0, mark it as X, go to the end, and write a new 0.
3. Return and do the same for the second 0 block (mark each used 0 as Y).
4. Restore $X \to 0$ and $Y \to 0$, then halt-accept.

Why this works: one new 0 is appended for every 0 in both blocks, so the last block has length $n + m$.

Part 2. Double any binary string: input w, output ww. Language accepted: $\{0,1\}^*$.

Simple TM idea:

1. Scan from left to right for an unmarked symbol.
2. If it is 0, mark it (say X), go to end, write 0; if it is 1, mark it (say Y), go to end, write 1.
3. Return to the left and repeat until all original symbols are marked.
4. Restore marks $X \to 0$, $Y \to 1$, then halt-accept.

Why this works: the machine copies each original symbol to the end in the same order, so final tape is exactly ww.

3.5.3 Homework 6: Turing Machines on Binary Strings (Exercise C)

Transition diagrams (simple/high-level) for the previous parts:

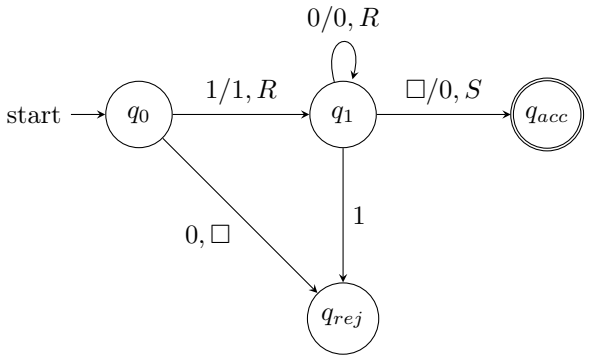

Figure 24: Exercise A1: accept 10^n and append one 0

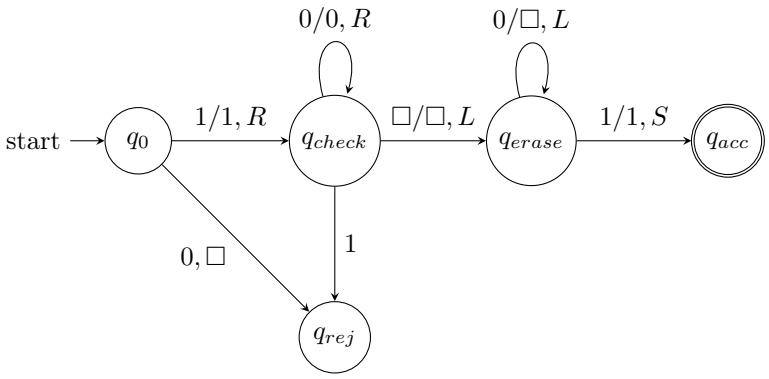

Figure 25: Exercise A2: accept 10^n and erase zeros to output 1

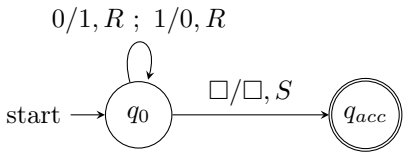

Figure 26: Exercise A3: swap every 0 and 1

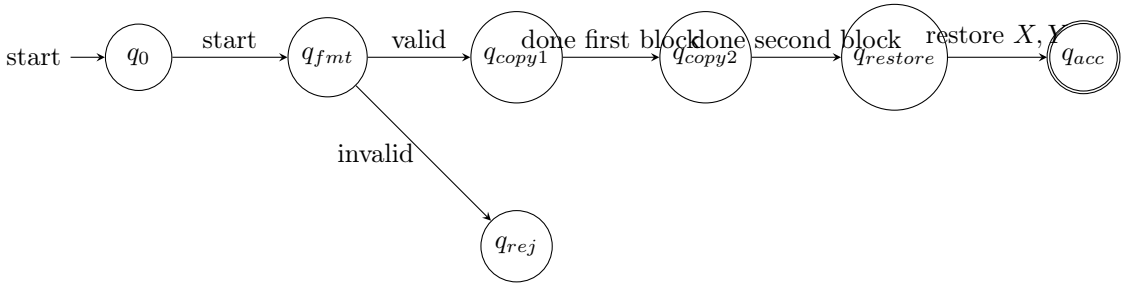

Figure 27: Exercise B1: unary addition $10^n10^m \mapsto 10^n10^m10^{n+m}$ (high-level flow)

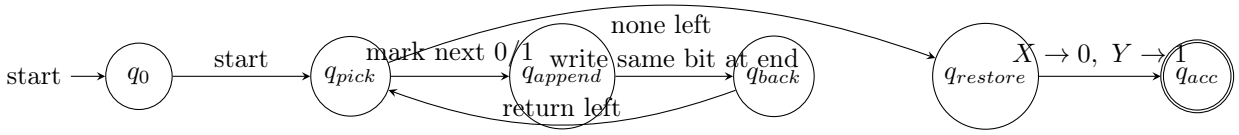

Figure 28: Exercise B2: doubling binary string $w \mapsto ww$ (high-level flow)

3.5.4 Homework 6: Turing Machines on Binary Strings (Exercise D)

Given prompt: accept binary strings that contain at least one 1, and do not halt.

Short note: With the standard TM definition, an *accept* state is halting. So "accept and never halt" is not possible in the strict sense.

The usual intended version is:

- If input has at least one 1, go to an accept state.
- If input has no 1 (only 0's or empty), loop forever.

Simple transition diagram (recognizer version):

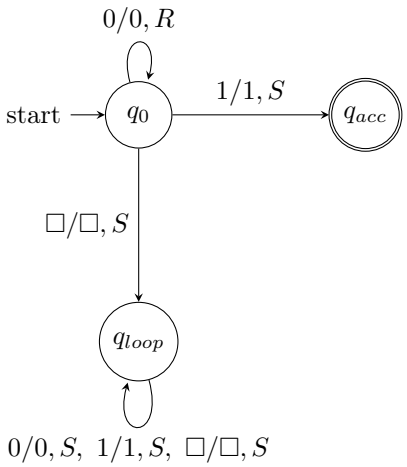

Figure 29: Exercise D: accepts strings with at least one 1, loops otherwise

3.5.5 Homework 6: Turing Machines on Binary Strings (Exercise E)

Part 1. Multitape TM: input $w \in \{0,1\}^*$, output 0^n1^m. Here, n is the number of 0's in w, and m is the number of 1's.

Simple 3-tape idea:

- Tape 1: input w (read only).
- Tape 2: counter/output for 0's (write one 0 each time Tape 1 reads a 0).
- Tape 3: counter/output for 1's (write one 1 each time Tape 1 reads a 1).

After Tape 1 ends, copy Tape 2 then Tape 3 to the final output tape. Result is 0^n1^m.

Part 2 (Challenge). Multitape TM: input 10^n10^m, output $10^n10^m10^{nm}$. Simple 4-tape idea:

- Tape 1: input 10^n10^m.
- Tape 2: store the first zero-block (n zeros).
- Tape 3: store the second zero-block (m zeros).
- Tape 4: build the product block.

Algorithm:

1. Validate format 10^n10^m and copy the two zero-blocks to Tapes 2 and 3.
2. For each 0 on Tape 2, scan Tape 3 once and append all its 0's to Tape 4.
3. When Tape 2 is finished, Tape 4 has exactly nm zeros.
4. Output $10^n10^m10^{nm}$ and halt-accept.

Why this works: repeating a copy of m zeros exactly n times gives nm zeros.

3.5.6 Homework 6: Problems on Decidability (Exercise 1)

For each statement, we say if it is true or false, with a short reason.

1. **True.** If L_1 and L_2 are decidable, then $L_1 \cup L_2$ is decidable.

   Run the decider for L_1 and the decider for L_2 on input w. Accept if at least one accepts; otherwise reject.

2. **True.** If L is decidable, then $\overline{L}$ is decidable.

   Just run the decider for L and flip the answer (accept $\leftrightarrow$ reject).

3. **True.** If L is decidable, then L^* is decidable.

   For input w of length n, check all possible ways to split w into pieces. There are finitely many splits. For each piece, run the decider for L. Accept if some split has all pieces in L; else reject.

4. **True.** If L_1 and L_2 are r.e., then $L_1 \cup L_2$ is r.e.

   Run both recognizers in dovetail (step-by-step). If either one accepts, accept.

5. **False.** If L is r.e., it does *not* imply $\overline{L}$ is r.e.

   Counterexample: $HALT = \{\langle M, w\rangle \mid M \text{ halts on } w\}$ is r.e., but $\overline{HALT}$ is not r.e.

6. **True.** If L is r.e., then L^* is r.e.

   A recognizer can enumerate all splits $w = x_1 \cdots x_k$ and dovetail runs of the recognizer for L on each x_i. If there is a valid split with all $x_i \in L$, it will eventually be found and accepted.

3.6 Week 7

3.6.1 Homework 7: Decidable vs r.e. vs co-r.e.

Given:

$L_1 := \{M \mid M \text{ halts on itself}\}, \quad L_2 := \{\langle M, w\rangle \mid M \text{ halts on } w\}, \quad L_3 := \{\langle M, w, k\rangle \mid M \text{ halts on } w \text{ in at most } k \text{ steps}\}.$

1) L_1

- **r.e.: yes.** Simulate M on input $\langle M\rangle$. If it halts, accept.
- **decidable: no.** This is a halting-type problem (same difficulty as $HALT$), so no decider exists.
- **co-r.e.: no.** If both L_1 and $\overline{L_1}$ were r.e., then L_1 would be decidable, contradiction.

2) L_2

- **r.e.: yes.** This is exactly $HALT$. Simulate M on w; if it halts, accept.
- **decidable: no.** $HALT$ is undecidable.
- **co-r.e.: no.** $\overline{HALT}$ is not r.e.

3) L_3

- **decidable: yes.** Simulate M on w for exactly k steps. Accept if halts within those steps; otherwise reject.
- **r.e.: yes.** Every decidable language is r.e.
- **co-r.e.: yes.** Every decidable language has decidable complement, hence co-r.e.

3.6.2 Homework 7: Asymptotic Notation (Exercise 2)

Let $f, g, h : \mathbb{N} \to \mathbb{R}_{\geq 0}$.

1. $f \in O(f)$.

   Take $c = 1$ and any n_0. Then $f(n) \leq c\, f(n)$ for all $n \geq n_0$.

2. If $c > 0$ is constant, then $O(c \cdot f) = O(f)$.

   Multiplying by a positive constant only rescales values. So any upper bound of the form $C\, cf(n)$ is still just a constant times $f(n)$, and vice versa. Therefore the two classes are equal.

3. If $f \leq g$ for large enough inputs, then $O(f) \subseteq O(g)$.

   Let $u \in O(f)$. Then $u(n) \leq Cf(n)$ eventually. Also $f(n) \leq g(n)$ eventually, so $u(n) \leq Cg(n)$ eventually. Hence $u \in O(g)$.

4. If $O(f) \subseteq O(g)$, then $O(f + h) \subseteq O(g + h)$.

   Since $f \in O(f)$ and $O(f) \subseteq O(g)$, we get $f \in O(g)$, so for some $A > 0$ and large n:

   $$f(n) \leq A\, g(n).$$

   If $u \in O(f + h)$, then for some $C > 0$ and large n:

   $$u(n) \leq C(f(n) + h(n)) \leq C(Ag(n) + h(n)) \leq C \max\{A, 1\}(g(n) + h(n)).$$

   So $u \in O(g + h)$.

5. If $h(n) > 0$ for all n and $O(f) \subseteq O(g)$, then $O(f \cdot h) \subseteq O(g \cdot h)$.

   As above, $f \in O(g)$, so $f(n) \leq Ag(n)$ eventually. Let $u \in O(fh)$. Then $u(n) \leq Cf(n)h(n) \leq CAg(n)h(n)$ eventually (valid since $h(n) > 0$). Hence $u \in O(gh)$.

3.6.3 Homework 7: Asymptotic Notation (Exercise 3)

Let $i, j, k, n \in \mathbb{N}$.

1. If $j \leq k$, then $O(n^j) \subseteq O(n^k)$.

   For $n \geq 1$, $n^j \leq n^k$. Apply Exercise 2(3).

2. If $j \leq k$, then $O(n^j + n^k) \subseteq O(n^k)$.

   For $n \geq 1$, $n^j + n^k \leq 2n^k$, so $n^j + n^k \in O(n^k)$. Therefore $O(n^j + n^k) \subseteq O(n^k)$.

3. Assume $a_i \geq 0$ for all i and $a_k > 0$. Then

   $$O\left(\sum_{i=0}^{k} a_i n^i\right) = O(n^k).$$

   For $n \geq 1$,

   $$\sum_{i=0}^{k} a_i n^i \leq \left(\sum_{i=0}^{k} a_i\right) n^k,$$

   so the polynomial is in $O(n^k)$. Also, since $a_k > 0$,

   $$a_k n^k \leq \sum_{i=0}^{k} a_i n^i,$$

   which gives the matching lower-order comparison, so the classes are equal.

4. $O(\log n) \subseteq O(n)$.

   For $n \geq 2$, $\log n \leq n$, so inclusion follows.

5. $O(n \log n) \subseteq O(n^2)$.

   For $n \geq 2$, $\log n \leq n$, hence $n \log n \leq n^2$.

3.6.4 Homework 7: Asymptotic Notation (Exercise 4)

Relationships between the classes:

1. $O(\sqrt{n}) \subsetneq O(n)$.

   Since $\sqrt{n} \leq n$ for $n \geq 1$, inclusion holds. It is strict because $n \notin O(\sqrt{n})$.

2. $O(n^2) \subsetneq O(2^n)$.

   Exponential growth dominates polynomial growth, so $n^2 \in O(2^n)$, and strictness holds since $2^n \notin O(n^2)$.

3. $O(\log n) \subsetneq O((\log n)^2)$.

   For $n \geq 2$, $\log n \leq (\log n)^2$. Strictness holds because $(\log n)^2 \notin O(\log n)$.

4. $O(2^n) \subsetneq O(3^n)$.

   Since $2^n \leq 3^n$, inclusion holds. Strictness holds because $3^n/2^n = (3/2)^n \to \infty$, so $3^n \notin O(2^n)$.

5. $O(\log_2 n) = O(\log_3 n)$.

   By change of base,

   $$\log_2 n = \frac{\log_3 n}{\log_3 2},$$

   so they differ only by a positive constant factor.

3.6.5 Homework 7: Asymptotic Notation (Exercise 5)

Classic sorting comparisons:

1. **Bubble sort vs Insertion sort**

   - Bubble sort: worst/average $\Theta(n^2)$, best $\Theta(n)$ only with early-stop optimization.
   - Insertion sort: worst/average $\Theta(n^2)$, best $\Theta(n)$ on already sorted input.

   So asymptotically they are the same class in worst case: both in $O(n^2)$.

2. **Insertion sort vs Merge sort**

   - Insertion sort: $\Theta(n^2)$ worst/average.
   - Merge sort: $\Theta(n \log n)$ worst/average/best.

   Merge sort is asymptotically better for large n because $n \log n$ grows slower than n^2.

3. **Merge sort vs Quick sort**

   - Merge sort: always $\Theta(n \log n)$.
   - Quick sort: average $\Theta(n \log n)$, worst $\Theta(n^2)$ (bad pivots).

   So in average case they are similar, but merge sort has the stronger worst-case guarantee. (In practice, quick sort is often very fast due to low constant factors and cache behavior.)

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, Title, Publisher, Year.